Nonlinear Model Order Reduction of Intrinsic Beam Equations via Quadratic-Bilinear POD

Pandey, Abhyuday
Student in MSc Aerospace Engineering

Under the supervision of:
Michalow, Maciej, Doctoral Fellow, ISAE-SUPAERO
Ribeiro-Lustosa, Leandro, Associate Professor in DCAS, ISAE-SUPAERO
RP title: Model reduction for very flexible aircraft flight simulator
ISAE-SUPAERO, Toulouse, France

Abstract

Geometrically exact intrinsic beam models are used extensively in the modelling of flexible aircraft, with the beam model often linearised due to the complexity of the full nonlinear model. This paper introduces a novel approach to reduce the model without linearisation, using QB-POD. The quadratic nonlinearity is recast as a matrix-vector involving the Kronecker product; this matrix is then reduced with POD, without online full-rank projection. The method is demonstrated on a finite-element model.
An analytical expression for the Jacobian of the intrinsic beam equations is also derived, and its development for a finite-element model is explained.

I. Introduction

High-altitude pseudo-satellite (HAPS) is a class of very high-endurance aircraft that can maintain self-sustained flight for long periods of time. Their applications include subservience and reconnaissance, radiocommunication, weather and environmental monitoring. To sustain flight for long periods, these aircraft often employ solar cells, which require a large surface and a means to store power at night. Weather balloons solve this problem by their neutral buoyancy and are a common aircraft in this class, but their drawback is that they are not manoeuvrable, which is often undesirable. In a traditional rigid-wing aircraft, the need for a large wing surface for solar cells would lead to a very large and heavy aircraft, which then, in turn, requires a lot of power and a large battery, leading to a vicious cycle and infeasibility. To have light wings with a large wing surface, one solution is to make the wing pliable - that is, flexible wings.

Flexible wings cause the aircraft to have poor handling qualities and need good control systems to make them human-flyable. The problem of controlling aircraft with flexible wings is not limited to the niche of HAPS - the newer generation of commercial passenger and cargo jets have more flexible wings (although to a much smaller extent) for efficiency gains[19]. Hence, progress in this field can be applicable more widely.

All aerofoils have the same basic feedback mechanism; the aerodynamic forces and control surfaces change the characteristics of the wing, which in turn change the aerodynamic forces. For very flexible wings, this effect is much more pronounced and requires special attention. Traditional aircraft can be quite accurately modelled as completely rigid aerofoils with constant aerodynamic coefficients. In flexible aircraft, the high-amplitude displacement of the aerofoils leads to large changes in the aerodynamic coefficients, which must be modelled correctly for robust control design. One approach to controlling such aircraft was investigated in [13], using model predictive control to solve an optimisation problem at each time step to find the best control inputs over a future time horizon, with moving horizon estimation to predict the future state.

Many of the most powerful control methods require an in-loop simulation of the system. Several tools exist which can model the dynamics of flexible aircraft. Two well-known examples are UM/NAST and SHARPy, both of which have been experimentally validated using data from the Pazy wing [12][9]. However, the full nonlinear models are large and cannot run in real-time on embedded hardware, and thus cannot be used for control methods that require online (real-time) simulation of the system, so a smaller model is needed.

Model reduction is a realm of mathematical tools which aims to speed up the online computation of a system, often adding some offline computation (done before the real-time, not recomputed at each time step).

II. Research gap

In his PhD thesis, Goizueta[16] developed reduced order models for flexible aircraft at low speeds starting from a full nonlinear aerodynamic model in UVLM and a geometrically exact intrinsic beam model. He used Krylov-subspace model reduction methods to reduce a linearised aerodynamic model and coupled it with a linearised beam model to get a reduced aeroelastic model.

In this paper, a Kronecker product-based formulation is used to express the quadratic nonlinear term of the geometrically exact intrinsic beam as a matrix product, such that POD can be directly on the nonlinear term with no need for online full-rank projections or linearization. First, all the mathematical derivations needed to apply this scheme on a finite-element model are developed and explained in general. Then the viability of the scheme is tested on a finite-element model of a geometrically exact intrinsic beam. As this is an intrusive reduction scheme, meaning it requires access to the governing equations and system matrices, a finite-element model of the nonlinear geometrically exact intrinsic beam model is built from scratch.

III. Literature review

The flexibility of the wings introduces three sources of nonlinearity for the full aircraft model: the structural model of the wing, the aerodynamic model, and the rigid-body dynamics. The focus in this paper is on reducing the structural model using an intrinsic beam formulation, modelling the wing's structural dynamics as a 1D beam in 3D space.

A. Intrinsic beam formulation

Linear beam theory has existed for many centuries and is used widely. Modelling a flexible aircraft undergoing large deformation, however, requires a geometrically exact structural model. Modern nonlinear beam theory stems from the work of Kirchhoff and Clebsch, employing an intrinsic formulation (not using position or finite rotation in the equations of motion). Such a formation requires kinematic relations and/or constitutive relations - equations relating physical quantities (for example, generalised strains to generalised forces) through material properties - to close the set of equations. Borri and Mantegazza first derived a full nonlinear geometrically exact intrinsic formulation in [1], but their formulation used displacement-based equations to close the system. This is undesirable as equations including displacements and finite-rotations have infinite-order (sinusoidal) nonlinearities. In [2], Hodges et al created a mixed formulation based on the intrinsic equations of motion and constitutive equations. While this model has low-order nonlinearities, the adjoined kinematical equations, a set of generalised strain-displacement equations, still led to sinusoidal nonlinearities. Then, in [4], Hodges et al derived a set of equations that completely omit displacement and finite-rotation, employing an intrinsic formulation for both the equations of motion as well as the kinematic relation (a set of generalised velocity-displacement equations). Displacements and finite-rotations can still be computed in post-processing and are often necessary for other modules (aerodynamics), but their omission in the beam equation limits the order of the non-linearities to two. The intrinsic equations of motion are given as

$$\dot{P} = -\tilde{\Omega}P + F' + (\tilde{\kappa} + \tilde{k})F + f \tag{1}$$

$$\dot{H} = -\tilde{\Omega}H - \tilde{V}P + M' + (\tilde{\kappa} + \tilde{k})M + (\tilde{e}_1 + \tilde{\gamma})F + m \tag{2}$$

where:

- F is the cross-sectional stress resultant force vector;
- M is the cross-sectional stress resultant moment vector;
- f is the external force;
- m is the external moment;
- γ is the deformed beam internal strain vector;
- κ is the deformed beam curvature and twist vector, P is the cross-sectional inertial linear momentum vector;
- H is the cross-sectional inertial angular momentum vector;
- V is the inertial linear vector of the deformed beam cross-sectional frame;
- Ω is the inertial angular vector of the deformed beam cross-sectional frame;
- $e_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^{\mathsf{T}}$ is the unit vector along the beam axis;
- k is the pre-twist of the beam at rest; and
- the tilde hat, $\tilde{x}$, denotes the cross product matrix operator.

The constitutive relations (generalised force-strain and generalised momentum velocity equation, re-

spectively) are given as

$$\begin{pmatrix}\gamma\\ \kappa\end{pmatrix} = \mathcal{C}\begin{pmatrix}F\\ M\end{pmatrix},\ \begin{pmatrix}P\\ H\end{pmatrix} = \begin{bmatrix}\mu\Delta & -\mu\tilde{x}_{\text{cg}}\\ \mu\tilde{x}_{\text{cg}} & I_{\text{3x3}}\end{bmatrix}\begin{pmatrix}V\\ \Omega\end{pmatrix} = \mathcal{M}\begin{pmatrix}V\\ \Omega\end{pmatrix} \tag{3}$$

where $\mathcal{C}, \mathcal{M}$ are the generalized stiffness and mass matrices respectively. Finally, the equations are closed with the intrinsic kinematic equations given as

$$\dot{\gamma} = V' + (\tilde{\kappa} + \tilde{k})V + (\tilde{e}_1 + \tilde{\gamma})\Omega \tag{4}$$

$$\dot{\kappa} = \Omega' + (\tilde{\kappa} + \tilde{k})\Omega. \tag{5}$$

These equations can be written in a compact form as in [5] as

$$\mathcal{M}\dot{\mathbf{x}}_1 - \mathbf{x}_2' - \mathrm{E}\mathbf{x}_2 + \mathcal{L}_1(\mathbf{x}_1)\mathcal{M}\mathbf{x}_1 + \mathcal{L}_2(\mathbf{x}_2)\mathcal{C}\mathbf{x}_2 = \mathbf{f}_{\text{ext}}, \tag{6}$$

$$\mathcal{C}\dot{\mathbf{x}}_2 - \mathbf{x}_1' + \mathrm{E}^{\mathsf{T}}\mathbf{x}_1 - \mathcal{L}_1^{\mathrm{T}}(\mathbf{x}_1)\mathcal{C}\mathbf{x}_2 = \mathbf{0}, \tag{7}$$

where

- $\mathbf{x}_1 = \begin{bmatrix}V & \Omega\end{bmatrix}^{\mathsf{T}}$, $\mathbf{x}_2 = \begin{bmatrix}F & M\end{bmatrix}^{\mathsf{T}}$, $\mathbf{f}_{\text{ext}} = \begin{bmatrix}f & m\end{bmatrix}^{\mathsf{T}}$
- $\mathcal{L}_1\left(\begin{pmatrix}a\\ b\end{pmatrix}\right) = \begin{bmatrix}\tilde{b} & 0\\ \tilde{a} & \tilde{b}\end{bmatrix}$, $\mathcal{L}_2\left(\begin{pmatrix}a\\ b\end{pmatrix}\right) = \begin{bmatrix}0 & \tilde{a}\\ \tilde{a} & \tilde{b}\end{bmatrix}$,
- $\mathrm{E} = \begin{bmatrix}\tilde{k} & 0\\ \tilde{e}_1 & \tilde{k}\end{bmatrix}$.

Note, that both $\mathcal{L}_i$ are linear mappings, as (for example)

$$\begin{aligned}
&\mathcal{L}_1(\lambda u + v)\cdot w\\
&= \begin{pmatrix}(\lambda u_2 + v_2)\times w_1\\ (\lambda u_1 + v_1)\times w_1 + (\lambda u_2 + v_2)\times w_2\end{pmatrix}\\
&= \begin{pmatrix}\lambda u_2 \times w_1\\ \lambda u_1 \times w_1 + \lambda u_2 \times w_2\end{pmatrix} + \begin{pmatrix}v_2 \times w_1\\ v_1 \times w_1 + v_2 \times w_2\end{pmatrix}\\
&= \lambda\mathcal{L}_1(u)\cdot w + \mathcal{L}_1(v)\cdot w.
\end{aligned}$$

This property is useful later in the paper when defining the finite element matrices.

B. Finite element discretization

Solving this set of beam equations analytically is practically impossible; it must instead be solved numerically. This has been done in the work of Artola [14], by putting the equations into the weak formulation, and integrating with a 3-node finite element with quadratic shape functions under the assumption that the mass and stiffness matrices remain constant within each element, leading to the following equation for each element:

$$\mathcal{M}_{\text{el}}\dot{\mathbf{V}}^h + \mathcal{K}_{\text{el, lin}}\mathbf{V}^h + \mathcal{K}_{\text{el, nl}}(\mathbf{V}^h)\mathbf{V}^h = \mathcal{F}^h, \tag{8}$$

where $\mathbf{V}^h = \begin{pmatrix}\mathbf{V}^1 & \mathbf{V}^2 & \mathbf{V}^3\end{pmatrix}^{\mathsf{T}}$ is the finite-element approximation of the state vector $\mathbf{V} = \begin{pmatrix}\mathbf{x}_1 & \mathbf{x}_2\end{pmatrix}^{\mathsf{T}}$ at the three nodes of the element.

C. Model reduction

The quality of a ROM can be measured by metrics such as relative error to the FOM solution and the speed-up. Speed-up is defined as

$$S = \frac{T_{FOM}}{T_{ROM}}, \tag{9}$$

so $S > 1$ means the ROM is S-times faster than the FOM. The numerical stability of the ROM is also of concern, as the reduced system must be solved numerically. The details of numerical stability of different root-finding algorithms is out of the scope of this paper. All static models will be solved with `scipy.fsolve`'s default solver and will be given the zero vector as the initial guess. Multi-fidelity approaches, as discussed in [17] are also outside the scope.

The method most commonly used to reduce structural models such as this one is normal modal decomposition and truncation, wherein the model is projected onto its eigenmodes and the higher frequency modes are truncated. In the process, the model becomes linear. This is great for ease of computation; however, the loss of the non-linear dynamics can lead to higher truncation error at larger deformations. [5] used nonlinear normal modes (NNMs) to capture the full dynamics. This method showed improved accuracy, but shows less promise for reduction, as with any Galerkin-projection-based approach, the nonlinear terms must be computed at full-rank and then projected onto the modes. Furthermore, unlike with other approaches, computing the NNMs is expensive and unlike linear modes, they are amplitude-dependent and must be recomputed online.

1. Proper orthogonal decomposition

Another commonly used reduction technique is proper orthogonal decomposition (POD) with Galerkin projection. The complete theory and process of computing a POD basis is well explained in [15, ch. 12] and [3], and is omitted here for brevity. What follows is a much abridged explanation of the most relevant concepts.

Many samples of the state vector are captured from the FOM,

$$\mathbf{X} = \begin{bmatrix}| & | & & |\\ x_1 & x_2 & \dots & x_m\\ | & | & & |\end{bmatrix},$$

where each x_k is a single sample. A modal basis can be calculated by applying singular value decomposition on $\mathbf{X}$, which can be ordered by the magnitude of the singular values and truncated up to a desired order, l, to obtain a reduced basis, Φ_l. This method is proper, meaning it guarantees that for a given l, Φ_l is the optimal l-rank basis for the $\|\cdot\|_2$-norm [3, p. 6]. The number of modes to include can be calculated using the retained-variance (RV) equation[7]

$$\frac{\sum_1^l \sigma_i}{\sum_m^l \sigma_i} > RV.$$

Any full rank vector (of rank n) can be reduced to rank l by

$$x \in \mathbb{R}^n \rightarrow \hat{x} = \Phi^{\mathsf{T}} x, \hat{x} \in \mathbb{R}^l \tag{10}$$

$$x = \Phi \hat{x} \tag{11}$$

such that a matrix operation with a matrix $A \in \mathbb{R}^{n\times n}$ can be reduced

$$y = Ax = A\Phi\hat{x} \tag{12}$$

$$\implies \hat{y} = \Phi^T A \Phi \hat{x} \tag{13}$$

$$\implies \hat{y} = \hat{A}\hat{x},\ \hat{A} \equiv \Phi^T A \Phi \hat{x} \in \mathbb{R}^{l\times l} \tag{14}$$

While this method has some of the same drawbacks due to Galerkin projection, the benefits are that modes are computed cheaply and offline, and being data-driven, if the samples cover the nominal range well, the reduced basis should perform well. POD can be coupled with a technique called discrete empirical interpolation method (DEIM), wherein the nonlinearity is sampled, and a similar process to POD is followed; projecting the nonlinearity onto its own SVD basis and using a greedy algorithm to compute the best points to interpolate the nonlinearity from[15]. This obviates the need to compute nonlinear terms at full-rank, leading to meaningful speed-up. Being data-driven, both POD and DEIM entail computationally heavy offline sample collection, which is done on the full-order model (FOM). The quality of these samples - whether they cover the whole range of expected dynamics well - has a critical impact on the quality of the reduced order models (ROMs)

2. QB-POD

The Kronecker product, denoted as $A \otimes B$, is a tensor product operation between matrices of arbitrary dimensions, resulting in a block matrix where each element of the first matrix multiplies the second,

$$A \otimes B = \begin{pmatrix} A_{11}B & A_{12}B & ... & A_{1n}B \\ ... & ... & & ... \\ A_{m1}B & A_{m2}B & ... & A_{mn}B \end{pmatrix}.$$

For vectors, the Kronecker product is the vectorised form of $x^T x$ with $x \in \mathbb{R}^n$. Its application for defining finite element matrices is not widespread, but it is well attested in the literature. In [11], they are used to quickly generate the stiffness matrix for FDM discretisation on a tensor grid. [8] uses the Kronecker product to represent polynomial nonlinearities of up to 3rd order (and their derivatives) as matrix products and even presents as an example a finite element implementation of Euler beam theory using Kronecker products. The system of interest in this paper has only second-order nonlinearities, hence the nonlinear term can be written as a quadratic form. The approach outlined in [10], which shall be referred to as Quadratic-bilinear POD (QB-POD), allows for the reduction of the nonlinear term with POD without online projection back to full-rank (as with any Galerkin-based approach, like POD) or hyper-reduction schemes such as DEIM. Online full-rank projections severely hinder the speed-up of POD-ROMs, as while the complexity of the linear term is reduced, the complexity of the higher-order terms remains the same. DEIM's method for avoiding full-rank projection by linearly interpolating the nonlinear term inevitably loses fidelity in nonlinear dynamics. DEIM also proves impractical (or at least involved and cumbersome) if the nonlinearity is not defined point-wise (such as with FEM where the value of the nonlinear term at one node depends on many nodes). In the QB-POD approach, the quadratic form is reduced onto the POD basis in the same manner as a linear operator, with no further approximation.

QB-POD requires writing the quadratic form as a matrix product with the Kronecker product of the state vector

$$\mathbf{f}_{\mathsf{NL}}(x) = \mathbf{H}\left(x \otimes x\right), \tag{15}$$

where $\mathbf{H} \in \mathbb{R}^{n\times n^2}$ is the matricized form of the 3rd order tensor representing the quadratic form. Another benefit of this formulation is that it makes the quadratic complexity of this operation explicit. A linear operation on an n-dimensional vector can be represented as an $(n \times n)$ matrix multiplying an n-row vector. A quadratic operation is represented by an $(n \times n^2)$ matrix multiplying an n^2-row vector.

Once the nonlinear term is in the Kronecker product form, it can directly be reduced like the matrices and vectors using the truncated POD basis as follows [10, p. 9]:

$$y = \mathbf{H}(x \otimes x) = \mathbf{H}(\Phi\hat{x} \otimes \Phi\hat{x}) \tag{16}$$

$$\implies \hat{y} = \Phi^t \mathbf{H}(\Phi \otimes \Phi)(\hat{x} \otimes \hat{x}) \tag{17}$$

$$\hat{y} = \hat{\mathbf{H}}(\hat{x} \otimes \hat{x}),\ \hat{\mathbf{H}} \equiv \Phi^T \mathbf{H}(\Phi \otimes \Phi) \in \mathbb{R}^{l\times l^2}, \tag{18}$$

hence the complexity of the quadratic from decreases

from n^2 to l^2. The Kronecker product formulation allows the 3rd order tensor representing the quadratic form to be recast as a matrix-vector product (with two copies of the state vector as the vector in that product), which enables standard linear algebra machinery to be applied directly. There is nothing special about quadratic forms; higher-order tensors could be recast similarly, although as the rest of the paper will amply prove, that could be quite cumbersome.

IV. Jacobian

In this section, the Jacobian of the intrinsic beam equations will be derived analytically. Having an expression for the Jacobian aids in the speed and stability of numerical solvers (especially for stiff problems such as ours). Note also, for the numerical solver, the quantity of interest is the Jacobian of the residue with respect to the state space of the principal variable $\mathbf{x}$ (not physical space). Starting by breaking up (8), (7) into the linear and nonlinear parts,

$$\mathbf{0} = -\begin{bmatrix} \mathcal{M} & \\ & \mathcal{C} \end{bmatrix} \dot{\mathbf{x}} + \begin{bmatrix} 0 & \mathbf{I} \\ \mathbf{I} & 0 \end{bmatrix} \mathbf{x}' + \begin{bmatrix} 0 & -\mathrm{E} \\ \mathrm{E}^{\mathsf{T}} & 0 \end{bmatrix} \mathbf{x} + \mathbf{f}_{\mathsf{NL}}(\mathbf{x}) \tag{19}$$

$$= \mathbf{f}_{\mathsf{LIN}}(\mathbf{x}) + \begin{pmatrix} \mathcal{L}_1(x_1)\mathcal{M}x_1 + \mathcal{L}_2(x_2)\mathcal{C}x_2 \\ -\mathcal{L}_1^{\mathsf{T}}(x_1)\mathcal{C}x_2 \end{pmatrix} \tag{20}$$

$$= \mathbf{f}_{\mathsf{LIN}}(\mathbf{x}) + \begin{bmatrix} \mathcal{L}_1(x_1)\mathcal{M} & \mathcal{L}_2(x_2)\mathcal{C} \\ & -\mathcal{L}_1^{\mathsf{T}}(x_1)\mathcal{C} \end{bmatrix} \cdot \mathbf{x} \tag{21}$$

$$= \mathbf{F}_t \cdot \dot{\mathbf{x}} + \mathbf{F}_{\mathsf{d}} \cdot \mathbf{x}' + \mathbf{F}_{\mathsf{E}} \cdot \mathbf{x} + \mathbf{F}_{\mathsf{NL}}(\mathbf{x}) \cdot \mathbf{x} \tag{22}$$

The Jacobian of $\mathbf{f}_{\mathsf{LIN}}(\mathbf{x})$ is trivial, especially after finite element discretisation, as both the time and spatial derivatives are expressed as linear operations on $\mathbf{x}$. The Jacobian of the nonlinear function $\mathbf{f}_{\mathsf{NL}}$ requires special attention due to the $\mathcal{L}_i$ functions. The derivative of these is easier to calculate in the form $\mathcal{L}_i(u)u$, assuming v is independent of u,

$$\nabla\left(\mathcal{L}_1(u)v\right)) \tag{23}$$

$$= \frac{d}{du}\left(\begin{bmatrix} \tilde{u_2} & \\ \tilde{u_1} & \tilde{u_2} \end{bmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix}\right) = \frac{d}{du}\begin{pmatrix} \tilde{u}_2 v_1 \\ \tilde{u}_1 v_1 + \tilde{u}_2 v_2 \end{pmatrix} \tag{24}$$

$$= \frac{d}{du}\begin{pmatrix} -\tilde{v}_1 u_2 \\ -\tilde{v}_1 u_1 - \tilde{v}_2 u_2 \end{pmatrix} = \begin{bmatrix} & -\tilde{v}_1 \\ -\tilde{v}_1 & -\tilde{v}_2 \end{bmatrix} \tag{25}$$

$$= -\mathcal{L}_2(v). \tag{26}$$

By similar manipulation, the following Jacobians can be found

$$\nabla\left(\mathcal{L}_2(u)v\right)) = -\mathcal{L}_1(v), \;\; \nabla\left(\mathcal{L}_1^{\mathsf{T}}(u)v\right)) = \mathcal{L}_1^{\mathsf{T}}(v) \tag{27}$$

This allows for a simple expression for the Jacobian of the nonlinear term to be found

$$\nabla \mathbf{f}_{\mathsf{NL}}(\mathbf{x}) \tag{28}$$

$$= \begin{bmatrix} \frac{d\mathcal{L}_1}{dx_1}\mathcal{M}x_1 + \mathcal{L}_1(x_1)\mathcal{M} & \frac{d\mathcal{L}_2}{dx_2}\mathcal{C}x_2 + \mathcal{L}_2(x_2)\mathcal{C} \\ -\frac{d\mathcal{L}_1^{\mathsf{T}}}{dx_1}\mathcal{C}x_2 & -\mathcal{L}_1^{\mathsf{T}}(x_1)\mathcal{C} \end{bmatrix} \tag{29}$$

$$= \begin{bmatrix} -\mathcal{L}_1(\mathcal{M}x_1) + \mathcal{L}_1(x_1)\mathcal{M} & -\mathcal{L}_2(\mathcal{C}x_2) + \mathcal{L}_2(x_2)\mathcal{C} \\ \mathcal{L}_1^{\mathsf{T}}(\mathcal{C}x_2) & -\mathcal{L}_1^{\mathsf{T}}(x_1)\mathcal{C} \end{bmatrix} \tag{30}$$

$$= \mathbf{F}_{\mathsf{NL}}(\mathbf{x}) - \begin{bmatrix} \mathcal{L}_2(\mathcal{M}x_1) & \mathcal{L}_1(\mathcal{C}x_2) \\ -\mathcal{L}_1^{\mathsf{T}}(\mathcal{C}x_2) & \end{bmatrix} \tag{31}$$

The Jacobian of the residue would be the Jacobian of the nonlinear term, plus $\mathbf{F}_{\mathsf{E}}$, plus the Jacobian of the derivative term - which depends on the finite difference scheme for spatial derivative - plus the Jacobian of the time derivative term - which depends on the time-marching scheme.

V. Intrinsic beam finite element model

At the outset, the goal was to use the structural solver implemented in UM/NAST, but at this preliminary stage of this body of work, it was not feasible to work with a black-box solver, as the reduction methods to be evaluated are intrusive - requiring direct access to the model. A toy model of an isotropic beam was therefore created in Python to facilitate this investigation. The structure is modelled with a finite-element (FE) model, with a simple strip-method aeroelastic module. The Python project can be accessed through GitHub[1]. For the purpose of comparison with a linear beam model, the project also contains an FE linear beam module as well as an analytical linear beam solver. To allow for comparisons between the linear beam model (which directly solves for displacement) and the nonlinear intrinsic beam model, a post-processor method has been implemented to calculate the displacement from strain and curvature. While it is possible to apply any boundary condition using the boundary condition matrix G_{BC} and function g_{BC} as described in [14], the default is cantilever conditions. This boundary condition is used for the remainder of this paper.

A. Setup and physical parameters

The span, L, of the beam is along the x-axis, the chord, c, is along y. The positive z-axis is vertically upwards. The physical parameters used for modelling are those of the Pazy wing (see [14, p. 157]), which

[1] https://github.com/abhyupandey21212/RP

has been widely used in the literature, so the results should be easy to compare and verify. Elements with 3 nodes are used, with quadratic shape functions defined on the domain $[0,1]$ corresponding to a scaling of the length of the element, h,

$$\phi_1(\sigma) = 2(\sigma - \tfrac{1}{2})(\sigma - 1) \tag{32}$$

$$\phi_2(\sigma) = -4\sigma(\sigma - \tfrac{1}{2}), \tag{33}$$

$$\phi_3(\sigma) = 2\sigma(\sigma - \tfrac{1}{2}). \tag{34}$$

B. Linear terms

While the work of derivation has already been done by Artola [14], it must be defined here with the Kronecker product to facilitate the computations required in the subsequent sections. The finite-element matrices of $\mathbf{F}_{\mathsf{E}}$ (and equivalently $\mathbf{F}_t$) for a single element can then be found by the following computation

$$\mathbf{F}_{\mathsf{E},el} \cdot \begin{pmatrix} x^1 \\ x^2 \\ x^3 \end{pmatrix} = h \begin{pmatrix} \sum_{j=1}^3 \int_0^1 \phi_1 \mathbf{F}_{\mathsf{E}} \phi_j x^j \, d\sigma \\ \sum_{j=1}^3 \int_0^1 \phi_2 \mathbf{F}_{\mathsf{E}} \phi_j x^j \, d\sigma \\ \sum_{j=1}^3 \int_0^1 \phi_3 \mathbf{F}_{\mathsf{E}} \phi_j x^j \, d\sigma \end{pmatrix} \tag{35}$$

$$= \mathbf{F}_{\mathsf{E}} \otimes h \begin{bmatrix} \int_0^1 \phi_1\phi_1 & \int_0^1 \phi_1\phi_2 & \int_0^1 \phi_1\phi_3 \\ \int_0^1 \phi_2\phi_1 & \int_0^1 \phi_2\phi_2 & \int_0^1 \phi_2\phi_3 \\ \int_0^1 \phi_3\phi_1 & \int_0^1 \phi_3\phi_2 & \int_0^1 \phi_3\phi_3 \end{bmatrix} \cdot \begin{pmatrix} x^1 \\ x^2 \\ x^3 \end{pmatrix} \tag{36}$$

Since the matrix of the integrals of the shape functions is a constant matrix, one can generally define that $\forall \mathbf{A}$ matrix such that $\mathbf{f}_A(\mathbf{x}) = \mathbf{A} \cdot \mathbf{x}$, the FE realisation of the function is the matrix

$$\mathbf{A}_{el} = \mathbf{A} \otimes \begin{bmatrix} \int_0^1 \phi_1\phi_1 & \int_0^1 \phi_1\phi_2 & \int_0^1 \phi_1\phi_3 \\ \int_0^1 \phi_2\phi_1 & \int_0^1 \phi_2\phi_2 & \int_0^1 \phi_2\phi_3 \\ \int_0^1 \phi_3\phi_1 & \int_0^1 \phi_3\phi_2 & \int_0^1 \phi_3\phi_3 \end{bmatrix} \tag{37}$$

$$= \mathbf{A} \otimes \frac{h}{15} \begin{bmatrix} 4 & 2 & -1 \\ 2 & 16 & 2 \\ -1 & 2 & 4 \end{bmatrix} = \mathbf{A} \otimes \mathcal{T}'_{\int \phi_i \phi_j}. \tag{38}$$

Similarly, $\forall \mathbf{A}$ matrix such that $\mathbf{f}_A(\mathbf{x}) = \mathbf{A} \cdot \mathbf{x}'$, the FE realization of the function is the matrix

$$\mathbf{A}_{el} = \mathbf{A} \otimes \begin{bmatrix} \int_0^1 \phi_1\phi_1' & \int_0^1 \phi_1\phi_2' & \int_0^1 \phi_1\phi_3' \\ \int_0^1 \phi_2\phi_1' & \int_0^1 \phi_2\phi_2' & \int_0^1 \phi_2\phi_3' \\ \int_0^1 \phi_3\phi_1' & \int_0^1 \phi_3\phi_2' & \int_0^1 \phi_3\phi_3' \end{bmatrix} \tag{39}$$

$$= \mathbf{A} \otimes \frac{h}{6} \begin{bmatrix} -3 & 4 & -1 \\ -4 & 0 & -4 \\ 1 & -4 & 3 \end{bmatrix} = \mathbf{A} \otimes \mathcal{T}_{\int \phi_i \phi_j'}. \tag{40}$$

These definitions allow for the element-wise FE equation of the beam to be written as

$$(\mathcal{T}_{\int \phi_i \phi_j} \otimes \mathbf{F}_t)\dot{\mathbf{V}}^h + (\mathcal{T}'_{\int \phi_i \phi_j'} \otimes \mathbf{F}_d)\mathbf{V}^h \tag{41}$$

$$+ (\mathcal{T}_{\int \phi_i \phi_j} \otimes \mathbf{F}_{\mathsf{E}})\mathbf{V}^h + \mathbf{F}_{\mathsf{NL},el}(\mathbf{V}^h) = \mathcal{F}^h. \tag{42}$$

The FE realisation of the nonlinear term requires a more involved derivation. While derivation has been carried out by Artola [14], that realisation is both tedious and not compatible with QB-POD. In the next section, the nonlinear term will be written as a Kronecker matrix-vector product, and then its FE realisation will be derived.

The process of construction of global FE matrices is well understood in the literature, and hence is omitted here (except for the nonlinear term, which once again requires special consideration and is discussed in the next section).

C. Nonlinear term

To facilitate the derivations in this section, it is immensely useful to first define some notation and helpful transformation matrices. Recall first the following definitions:

$$x = \begin{pmatrix} x_1 & x_2 \end{pmatrix}^{\mathsf{T}} = \begin{pmatrix} V & \Omega & F & M \end{pmatrix}^{\mathsf{T}} \tag{43}$$

$$x_1 \otimes x_1 = \begin{pmatrix} V \\ \Omega \end{pmatrix} \otimes \begin{pmatrix} V \\ \Omega \end{pmatrix}, x_1 \otimes x_2 = \begin{pmatrix} V \\ \Omega \end{pmatrix} \otimes \begin{pmatrix} F \\ M \end{pmatrix}, ... \tag{44}$$

A common problem when using the Kronecker product is the transformation

$$\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \otimes \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \to \begin{pmatrix} u_1 \otimes v_1 \\ u_1 \otimes v_2 \\ u_2 \otimes v_1 \\ u_2 \otimes v_2 \end{pmatrix},$$

as the more natural way to define the quadratic nonlinearity would be to use the right-hand side realisation, but to be compatible with QB-POD, it must be in the left-hand realisation. To facilitate this, the following transformation matrices are defined

$$\begin{pmatrix} u_1 \otimes v_1 \\ u_1 \otimes v_2 \\ u_2 \otimes v_1 \\ u_2 \otimes v_2 \end{pmatrix} = K_1 \left(\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \otimes \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \right), \quad u_i, v_i \in \mathbb{R}^3,$$

$$\begin{pmatrix} x_1 \otimes x_1 \\ x_1 \otimes x_2 \\ x_2 \otimes x_1 \\ x_2 \otimes x_2 \end{pmatrix} = K_2 \left(x \otimes x \right), \quad x \in \mathbb{R}^{12},$$

$$\begin{pmatrix} V^1 \otimes V^1 \\ V^1 \otimes V^2 \\ \dots \\ V^3 \otimes V^2 \\ V^3 \otimes V^3 \end{pmatrix} = K_3(\mathbf{V}^h \otimes \mathbf{V}^h), \mathbf{V}^h \in \mathbb{R}^{36}.$$

Using the following identities

$$Ax \otimes By = (A \otimes B)(x \otimes y), \quad x, y \in \mathbb{R}^n,\ A, B \in \mathbb{R}^{n \times n},$$

$$\begin{aligned} \tilde{x}y &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} (x \otimes y) \\ &\equiv \mathcal{X}(x \otimes y), \end{aligned}$$

the functions $\mathcal{L}_1$, $\mathcal{L}_2$, $\mathcal{L}_1^T$ can be rewritten as

$$\begin{aligned} &\mathcal{L}_1\left(\begin{pmatrix} a \\ b \end{pmatrix}\right) \cdot \begin{pmatrix} c \\ d \end{pmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathcal{X} & \mathbf{0} \\ \mathcal{X} & \mathbf{0} & \mathbf{0} & \mathcal{X} \end{bmatrix} \begin{pmatrix} a \otimes c \\ a \otimes d \\ b \otimes c \\ b \otimes d \end{pmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathcal{X} & \mathbf{0} \\ \mathcal{X} & \mathbf{0} & \mathbf{0} & \mathcal{X} \end{bmatrix} K_1 \left(\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix}\right) \\ &= \mathbf{L_1}\left(\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix}\right), \ \ \mathbf{L}_1 \in \mathbb{R}^{6 \times 6^2}; \end{aligned}$$

$$\mathcal{L}_2\left(\begin{pmatrix} a \\ b \end{pmatrix}\right) \cdot \begin{pmatrix} c \\ d \end{pmatrix} = \mathbf{L}_2\left(\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix}\right), \ \ \mathbf{L}_2 \in \mathbb{R}^{6 \times 6^2}.$$

$$\mathcal{L}_1^T\left(\begin{pmatrix} a \\ b \end{pmatrix}\right) \cdot \begin{pmatrix} c \\ d \end{pmatrix} = \mathbf{L}_1^\intercal\left(\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix}\right), \ \ \mathbf{L}_1^\intercal \in \mathbb{R}^{6 \times 6^2},$$

Note that the superscript in $\mathbf{L}_1^\intercal$ does not denote the transpose of $\mathbf{L}_1$, it is the matricized form of $\mathcal{L}_1$, and it must be found independently. Importantly, if inconveniently, it is not equal to the transpose, $\mathbf{L}_1^\intercal \neq \mathbf{L}_1^T$. Using the previously defined nonlinear term $\mathbf{f}_{\text{NL}}$, the Kronecker form expression is then found as

$$\mathbf{f}_{\text{NL}}(x) \tag{45}$$

$$= \begin{pmatrix} \mathcal{L}_1(x_1)\mathcal{M}x_1 + \mathcal{L}_2(x_2)\mathcal{C}x_2 \\ -\mathcal{L}_1^\intercal(x_1)\mathcal{C}x_2 \end{pmatrix} \tag{46}$$

$$= \begin{bmatrix} \mathbf{L}_1 & \mathbf{L}_2 & \\ & & -\mathbf{L}_1^\intercal \end{bmatrix} \begin{pmatrix} x_1 \otimes \mathcal{M}x_1 \\ x_2 \otimes \mathcal{C}x_2 \\ x_1 \otimes \mathcal{C}x_2 \end{pmatrix} \tag{47}$$

$$= \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} & \mathbf{L}_2 \\ \mathbf{0} & -\mathbf{L}_1^\intercal & \mathbf{0} \end{bmatrix} \begin{pmatrix} (\mathbf{I}_{6\times 6} \otimes \mathcal{M})(x_1 \otimes x_1) \\ (\mathbf{I}_{6\times 6} \otimes \mathcal{C}(x_1 \otimes x_2) \\ (\mathbf{I}_{6\times 6} \otimes \mathcal{C})(x_2 \otimes x_2) \end{pmatrix} \tag{48}$$

$$= \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} & \mathbf{L}_2 \\ \mathbf{0} & -\mathbf{L}_1^\intercal & \mathbf{0} \end{bmatrix} \begin{bmatrix} \star \end{bmatrix} \begin{pmatrix} x_1 \otimes x_1 \\ x_1 \otimes x_2 \\ x_2 \otimes x_1 \\ x_2 \otimes x_2 \end{pmatrix} \tag{49}$$

$$= \begin{bmatrix} \mathbf{L}_1 & \mathbf{0} & \mathbf{L}_2 \\ \mathbf{0} & -\mathbf{L}_1^\intercal & \mathbf{0} \end{bmatrix} \begin{bmatrix} \star \end{bmatrix} K_2(x \otimes x) \tag{50}$$

$$\equiv \mathbf{H}(x \otimes x), \ \begin{bmatrix} \star \end{bmatrix} = \begin{bmatrix} \mathbf{I} \otimes \mathcal{M} & & \\ & \mathbf{I} \otimes \mathcal{C} & \\ & & \mathbf{0} \quad \mathbf{I} \otimes \mathcal{C} \end{bmatrix}. \tag{51}$$

The full realisation of some of the matrices K_i is impractical to include here due to their size; they can be found on GitHub[2]. The Jacobian of the nonlinear term could be expressed as a Kronecker matrix-vector product by following the same procedure as above on (31). However, by using the properties of derivatives of the Kronecker product as proved in [6], this computation can be much simplified,

$$\nabla \mathbf{f}_{\text{NL}}(x) = \nabla(\mathbf{H}(x \otimes x)) \tag{52}$$

$$= \mathbf{H}((\mathbf{I} \otimes x) + (x \otimes \mathbf{I})), \tag{53}$$

where $\mathbf{I} \in \mathbb{R}^n$, the same space as x. Note, however, that this definition of the Jacobian, while compact, is impractical. When solving numerically (for example, with `scipy.fsolve`), the Kronecker products here do not play nice due to the time constraints on Jacobian calls. It is therefore practical to define $\mathbf{H}_{3D} \in \mathbb{R}^{n \times n \times n}$ as a 3rd order tensor form of $\mathbf{H} \in \mathbb{R}^{n \times n^2}$, then using index notation the Jacobian can be expressed as a sum

$$[\nabla \mathbf{f}_{\text{NL}}]_{ik} = \sum_j \sum_l [\mathbf{H}_{3D}]_{ijl}(\delta_{jk}x_l + x_j\delta_{lk}) \tag{54}$$

$$= \sum_j [\mathbf{H}_{3D}]_{ikj}x_j + \sum_j [\mathbf{H}_{3D}]_{ijk}x_j, \tag{55}$$

which can be solved efficiently (using `scipy.einsum`), speeding up the computation dramatically. In fact, in general, when solving numerically, all Kronecker products should be solved this way for efficiency gains.

Now that the nonlinear term has been expressed as a matrix-vector product, it must be adapted for use with the finite element framework. Going back to the definition, the nonlinear term at k-th node of an

[2] https://github.com/abhyupandey21212/RP

element is

$$\mathbf{F}_{\text{NL},el}(\mathbf{V}^h)_k = h\int_0^1 \phi_k \mathbf{H}\left(\mathbf{V}^h \otimes \mathbf{V}^h\right) d\sigma \tag{56}$$

$$= h\int_0^1 \phi_k \mathbf{H}\left(\sum_l \phi_l V^l \otimes \sum_m \phi_m V^m\right) d\sigma \tag{57}$$

$$= h\int_0^1 \phi_k \mathbf{H}\left(\sum_l \sum_m \phi_l \phi_m (V^l \otimes V^m)\right) d\sigma. \tag{58}$$

Using the commutativity of integrals and sums for a continuous function, this can be converted into a matrix expression

$$(56) = \begin{bmatrix} \int_0^1 \phi_k\phi_1\phi_1\mathbf{H} & \int_0^1 \phi_k\phi_1\phi_2\mathbf{H} & \dots \end{bmatrix} \begin{pmatrix} V^1 \otimes V^1 \\ V^1 \otimes V^2 \\ \dots \\ V^3 \otimes V^3 \end{pmatrix}$$

$$= \left(\begin{bmatrix} \int_0^1 \phi_k\phi_1\phi_1 & \dots & \int_0^1 \phi_k\phi_3\phi_3 \end{bmatrix} \otimes \mathbf{H}\right) \begin{pmatrix} V^1 \otimes V^1 \\ \dots \\ V^3 \otimes V^3 \end{pmatrix}.$$

Define the constant matrix $\mathcal{T}_{\phi_i\phi_j\phi_k}$ as

$$\mathcal{T}_{\phi_i\phi_j\phi_k} = \begin{bmatrix} \int_0^1 \phi_1\phi_1\phi_1 & \int_0^1 \phi_1\phi_1\phi_2 & \dots & \int_0^1 \phi_1\phi_3\phi_3 \\ \int_0^1 \phi_2\phi_1\phi_1 & \int_0^1 \phi_2\phi_1\phi_2 & \dots & \int_0^1 \phi_2\phi_3\phi_3 \\ \int_0^1 \phi_3\phi_1\phi_1 & \int_0^1 \phi_3\phi_1\phi_2 & \dots & \int_0^1 \phi_3\phi_3\phi_3 \end{bmatrix}$$

$$= \frac{h}{210}\begin{bmatrix} 39 & 20 & -3 & 20 & 16 & -8 & -3 & -8 & -3 \\ 20 & 16 & -8 & 16 & 192 & 16 & -8 & 16 & 20 \\ -3 & -8 & -3 & -8 & 16 & 20 & -3 & 20 & 39 \end{bmatrix}.$$

Hence, finally, the FE realisation of the nonlinear term for a single element using the quadratic matrix is

$$\mathbf{F}_{\text{NL},el}(\mathbf{V}^h) = \left(\mathcal{T}_{\phi_i\phi_j\phi_k} \otimes \mathbf{H}\right) K_3 \left(\mathbf{V}^h \otimes \mathbf{V}^h\right). \tag{59}$$

Similarly to how global matrices are constructed for linear terms, the global matrix of the nonlinear term is constructed by overlapping the element matrices at shared nodes. However, this is slightly more involved for these matrices due to their shapes. This is illustrated in Fig. 1 for 3-node elements. This also illustrates why it might be easier to define a global

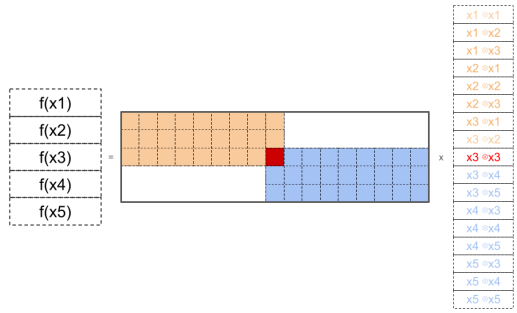

Figure 1 Illustration of global FE matrix with 2 elements, 5 nodes

matrix K_4 that converts $\mathbf{V} \otimes \mathbf{V}$ into

$$K_4(\mathbf{V} \otimes \mathbf{V}) = \begin{pmatrix} V^1 \otimes V^1 \\ V^1 \otimes V^2 \\ \dots \\ V^3 \otimes V^2 \\ V^3 \otimes V^3 \\ V^3 \otimes V^4 \\ V^3 \otimes V^5 \\ \dots \\ V^n \otimes V^n \end{pmatrix}, \tag{60}$$

where n is the number of nodes and $\mathbf{V} \in \mathbb{R}^n$ is the global FE state vector (all the nodal vectors stacked). This is similar to the function of K_3 except it acts on the global state vector instead of the element state vector. Let

$$\mathbf{H}_{el} = \mathcal{T}_{\phi_i\phi_j\phi_k} \otimes \mathbf{H}, \tag{61}$$

and let $\mathbf{H}_{gl}$ be the global nonlinear matrix constructed by overlapping the element matrices as illustrated in Fig. 1, then

$$\mathbf{F}_{\text{NL},gl}(\mathbf{V}) = \mathbf{H}_{gl} K_4(\mathbf{V} \otimes \mathbf{V}). \tag{62}$$

The matrix K_4, while simple in concept, is both very large (for a 15-node model, it has 8208 by 32400 entries). The each $(12 \cdot 6 \times 12 \cdot 6$ block in visualization in Fig. 2 corresponds to $x_{aj} \otimes x_{bk}$ with $a, b \in \{1, 2\}$ and $j, k \in \{1, 2, ..., 5\}$. Using SciPy's sparse matrix tools is recommended to avoid memory issues. It is also tedious and cumbersome to define for an arbitrary number of nodes, n. An implementation is included in the GitHub repository as a method of the `beamNL` class. The same method can be followed to find the global Jacobian matrix of the nonlinear term.

D. Validation

To check the validity of the approach of defining FE matrices with the Kronecker product, all the matrices were defined using both approaches: the approach

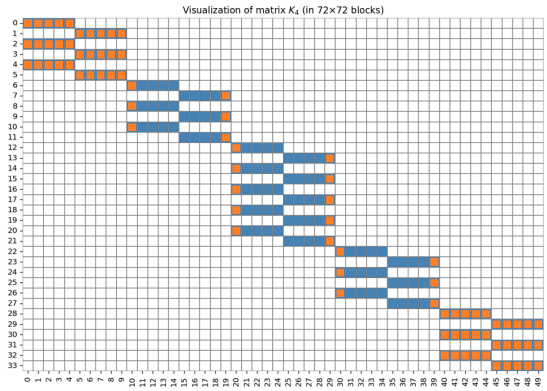

Figure 2 Visualisation of matrix K_4 for a 5-node 2-element model (the orange marks the columns removed to construct matrix K_5)

as detailed in [14] in Appendix C, and the one with the Kronecker product (as stated in (40) and (51)). For the sake of brevity, the former shall be referred to as the standard formulation and the latter as the Kronecker formulation. Both approaches gave identical matrices for linear terms. In the standard formulation for the nonlinear term, the matrix is only defined for a given state vector, while the Kronecker formulation gives a bigger but constant matrix. Hence, the matrices are of course not identical, but the error of the numerical evaluations of the nonlinear term for 1000 randomly generated state vectors between the two approaches was on the order of 10^{-15}. The Kronecker formulation is far more computationally efficient for the nonlinear term, requiring about 10 times less time for each computation. Solving each static problem took on the order of 10^{-2}s as compared to 10^{-1}s for the standard formulation (with the analytical Jacobian implemented).

As part of the validation, this FE model was compared to a linear Euler beam (whose deflection for a tip force under cantilever conditions can be analytically found). For tip force below 50 N, the two models gave very similar deflections, and if the nonlinear term was removed, the results were nearly identical (the error was on the order of 10^{-6}). This validates that the full order finite element model is correctly implemented, and reflects that the nonlinear term has a negligible contribution under small loads, which is as expected. The nonlinear beam model was also validated to be numerically stable for large loads, giving large deflections (up to 10^5 N).

A mesh convergence study was performed, shown in Fig. 3, which showed the relative error (in tip deflection under tip force in cantilever conditions) drops below 5% at 13 nodes (relative to a 35-node mesh, for 1000 random tip forces).

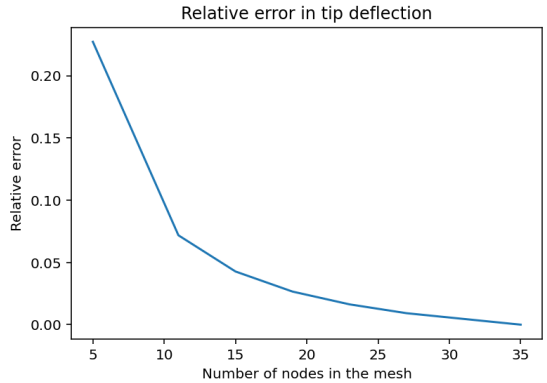

Figure 3 Mesh convergence study, FOM

To validate and measure the speed-up from the implementation of the Jacobian, different sizes of meshes were tested, as shown in Fig. 4, with and without the analytical Jacobian implemented. When no Jacobian is provided, the solver approximates it with finite-difference (FD). Implementing the Jacobian function led to a speed-up of $S \approx 10$.

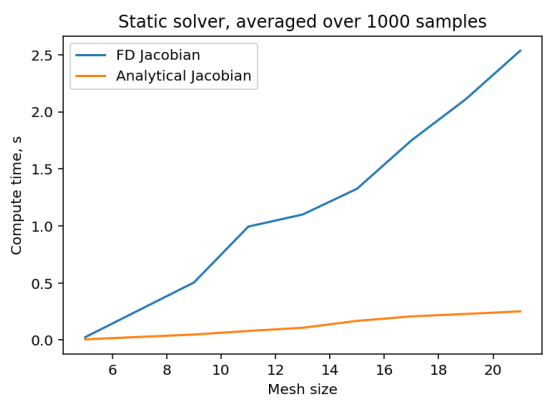

Figure 4 Computation time for different-sized meshes of FOM

VI. Proper orthogonal decomposition

The full order FE model with 15 nodes (7 elements) is used to capture 10000 samples of the beam under random vertical tip force (hence the ROMs are valid only for vertical tip loads). For a better model, a wider range of loads (perhaps completely random noise) should be captured, but this limited sample will suffice for the purpose of this paper.

A. Traditional POD

In this section, traditional POD is applied to the Kronecker formulation of the beam model. Meaning, the nonlinear term was calculated in the full-rank projection of the POD state vector.

In general, the treatment of boundary conditions for model reduction is straightforward for homogeneous boundary conditions, but not so for inhomogeneous boundary conditions. The cantilevered beam has homogeneous Dirichlet conditions on the clamped end (which can be expressed as a condition on $x_1(0) = \begin{pmatrix} V(0) & \Omega(0) \end{pmatrix}^T$) and inhomogeneous Neumann conditions at the free end on the beam's displacement. However, another benefit of the intrinsic formulation is that the inhomogeneous Neumann condition becomes a homogeneous Dirichlet condition on $x_2(L) = \begin{pmatrix} F(L) & M(L) \end{pmatrix}^T$[18, p. 343]. Homogeneous Dirichlet boundary conditions imply that the energy of those nodes is identically zero. Hence, these boundary conditions are applied by removing those nodes from the samples. Thus, the POD-ROM will compute only the "interior" of the model. Implementing this is straightforward for the matrices of the linear terms, if the boundary condition is on index k of the state vector, row and column k are removed from the matrix before the reduction step in (14), referred to henceforth as the interior-matrices. Since in traditional POD, the nonlinear term is calculated at full-rank, the boundary conditions are applied as in the FOM.

The stability and speed of both the ROM can also be aided by providing the Jacobian to the numerical solver. A Jacobian for the ROM can be found by following the same procedure as for the governing equations on the Jacobian: applying (14) on the interior-matrices for the linear terms, and replacing K_4 with K_5 in (62) for the nonlinear term. Comparing many mesh sizes for POD is time consuming, as samples for each must first be collected, so Fig. 4 shows only the speed-up for different sizes of meshes for the FOM. For the 3 mesh sizes (9-node, 15-node, and 21-node) for which samples were collected and a POD basis was computed, the average speed-up was $S = 5.7$, $S = 4.6$, and $S = 4.9$, respectively.

The POD basis is computed using SVD on the "interior" of the samples, and the energy of the first 20 modes [15, p. 459] is visualised in Fig. 5. With just 5 modes, $RV = 97\%$ is met, and with the first 10 $RV = 99.99\%$. For a linear system, this would imply that a very low-rank would perform very well. Indeed, if the nonlinear term is removed, a rank-2 POD-ROM performs well, giving an average speed-up - as defined in (9) - of $S \approx 150$ (see Fig. 6) and relative error on the order of 10^{-8}. Of course, removing

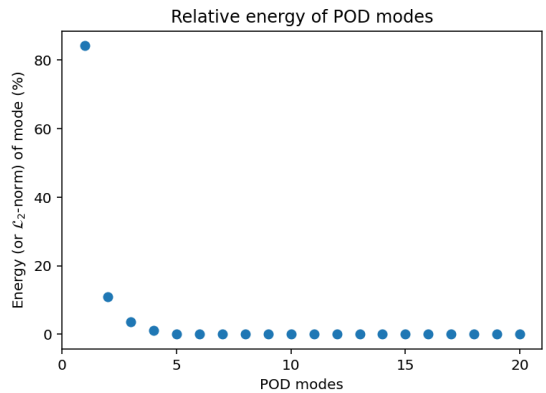

Figure 5 Relative energy of each POD mode

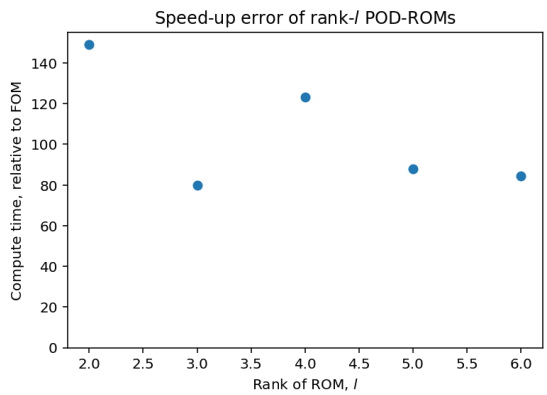

Figure 6 Speed-up for small loads (negligible nonlinear contribution)

the nonlinear term makes the model inaccurate for large displacements, and indeed the model struggles to converge for tip forces much larger than 50 N. In fact, the nonlinear term does not need to be removed at all, as under such small loads, its contribution is negligible, and the 2-mode POD-ROM still has the same stability.

This was not the case for larger, more representative load cases. The ROM was very numerically unstable for smaller values of l (converging in less than 10% of trials), and for larger values, the ROM was very slow, considerably slower than the FOM.

B. QB-POD

Due to the Kronecker product formulation, applying the homogeneous Dirichlet boundary conditions on the matricized nonlinear term requires an involved "de-interlacing" operation. Starting from matrix K_4 (as defined in (60)), all its columns corresponding to

the boundary condition indices are removed to define matrix K_5. Replacing K_4 with K_5 in (62) gives the matricized nonlinear term with the boundary condition indices removed.

As explained in the Literature Review, QB-POD allows for the reduction of the matricized quadratic nonlinearity without projection onto the full-rank space. This proved much more successful, but finding a good ROM entails finding a balance between reduced size and numerical stability. Even a rank-20 QB-POD-ROM is too numerically unstable, meaning the numerical solver either does not converge or has to do so many iterations as to negate the speed-up from the reduction.

As in the previous section, implementing the Jacobian leads to a similar $S \approx 10$ speed-up for the same rank ROM. All values presented in graphs or otherwise are collected with the Jacobian implemented. Even with the Jacobian, too few POD modes do not capture the nonlinearity and make the numerical solver unstable, leading to no convergence. As l is increased, the solver becomes more stable, but the model becomes larger. There is a Goldilocks zone of enough POD modes to make the solver stable enough to converge within a few iterations, while not so many modes that each iteration is too slow, such that the QB-POD-ROM can be meaningfully faster than FOM. This speed-up is more pronounced for bigger systems.

Take, for example, a 21-node, 10-element beam. Since each node has 12 degrees of freedom, this model is of rank-252. A sweep of different values of l was performed for 9, 11, 13, 17, and 19-node beams, first a coarse sweep, followed by a finer sweep in the interval of best performance. Fig. 7 and Fig. 8 show only the results of the 15-node and 21-node beams to avoid clutter.

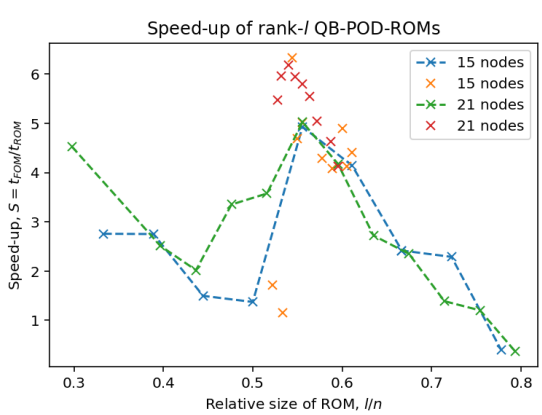

Figure 7 Speed-up of l-rank QB-POD-ROMs (averaged over 500 samples)

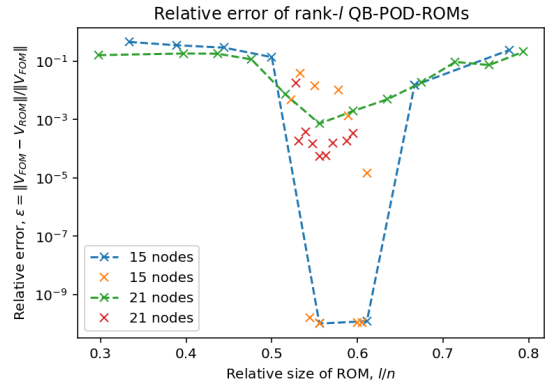

Figure 8 Relative error of l-rank QB-POD-ROMs (averaged over 500 samples)

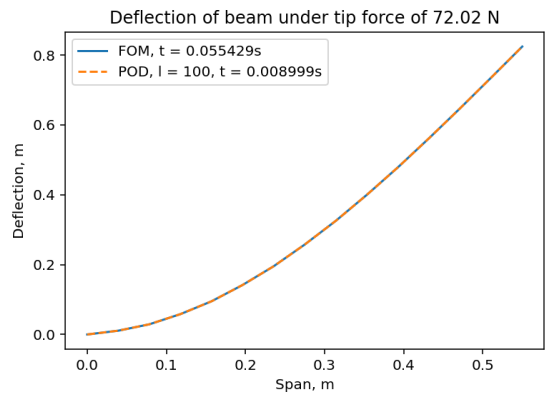

Figure 9 Example of a speed-up with rank-98 QB-POD-ROM for a static tip force problem

The speed-up starts high and drops initially before increasing and then drops off again. The best performing ROM for the 21-node beam was $l = 136$, which has a speed-up of $S = 6$ and a relative error on the order of 10^{-4}. For the 15-node beam, it was $l = 98$, and a relative error on the order of 10^{-9}. For both k-node beams (and all other values of k tested), the value of l that gave the best speed-up also had the lowest error, likely because it better represents the real model and converges in fewer iterations and with a smaller error.

The high speed-up at low l should be ignored; the ROMs with $\frac{l}{n} < 0.45$ have high relative error. The subsequent increase and drop show the phenomenon of a goldilocks zone. As l increases, the QB-POD-ROM becomes more stable and hence faster, but as l keeps increasing, the ROM becomes so large that it becomes slower again. Across all the k-node beams tested, the best ROM had $\frac{l}{n} \approx 0.55$, and the best speep-up and lowest error ROMs coincided.

A phenomenon that seems counterintuitive is the high error for large $\frac{l}{n}$. This is explained by the time limit imposed by `scipy.fsolve` by default, which times out for these ROMs, leading them not to converge.

VII. Further work

The natural next step from the work done in this paper is to implement a dynamic solver and apply the same reduction technique to it. Due to time constraints. The implementation has already been done in the `beamNL.dynamic_solver` method. It is conceptually close to the static solver, $\frac{d\mathbf{V}}{dt}$ is equal to $-1 \cdot \mathcal{M}_{gl}^{-1}$ times the residual of the static solver, and the Jacobian is found by scaling the Jacobian of the residual identically. The QB-POD solver is also implemented, similar to the FOM, it uses the residual of the QB-POD static solver and QB-POD Jacobians, scaled by $-1 \cdot \mathcal{M}_{gl,r}^{-1}$. Using the Radau solver in `scipy.integrate.solve_ivp` is was used.

Due to how similar the two solvers are, the same POD basis (from the samples of the static problem) can be used for both the static and dynamic solvers. The preliminary investigation showed promising results. The QB-POD-ROM showed an $S \approx 100$ speed-up compared to the FOM dynamic solver. However, more work still needs to be done on the dynamic solvers as they regularly diverge to infinity even for quite short time horizons (it was stable for up to 0.05s). Unfortunately, due to time constraints, this has not yet been addressed. Future work could start here and investigate the QB-POD of the dynamic solver.

The static beam solver can be coupled with a linear (strip) aerodynamic solver to give a basic aeroelastic model. This too has already been implemented, but joint reduction of the coupled system was not attempted due to time constraints.

The results of the previous section show that while the vast majority of the energy is contained within the first few modes, many more modes must be included to capture the nonlinear dynamics. This raises the question: Is it possible to reduce the linear and nonlinear terms to different sizes? One solution to this question is to use DEIM. Multi-fidelity models may also be relevant.

VIII. Conclusion

This paper introduces two novel developments for geometrically exact intrinsic beam modelling. An analytical derivation of the Jacobian, both in the strong form and for an arbitrary finite element model. This alone gives a speed-up for $S = 10$ for a static problem for a 15-node 7-element beam model, and shows a similar trend for other k-node beams, with a slight increase in the speed-up with k. The Kronecker formulation was used to express the quadratic nonlinearity as a matrix-vector product between an $n \times n^2$ matrix and the Kronecker product of the state vector. This allows for direct application of proper orthogonal decomposition for reduction, which gives a further $S \approx 6$ speed-up for the many different k-node beams. The QB-POD-ROM with the highest speed-up consistently has the lowest relative error, and has $\frac{l}{n} \approx 0.55$.

References

[1] M. Borri and P. Mantegazza, "Some contributions on structural and dynamic modeling of helicopter rotor blades", *L'Aerotecnica Missili e Spazio*, vol. 64, no. 9, pp. 143–154, 1985.

[2] D. H. Hodges, "A mixed variational formulation based on exact intrinsic equations for dynamics of moving beams", *International Journal of Solids and Structures*, vol. 26, no. 11, pp. 1253–1273, 1990, ISSN: 0020-7683. DOI: https://doi.org/10.1016/0020-7683(90)90060-9. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0020768390900609.

[3] S. Volkwein, "Model reduction using proper orthogonal decomposition", Jan. 2008.

[4] D. H. Hodges, "Geometrically exact, intrinsic theory for dynamics of curved and twisted anisotropic beams", *AIAA Journal*, vol. 47, no. 5, pp. 1308–1309, 2009. DOI: 10.2514/1.40556. eprint: https://doi.org/10.2514/1.40556. [Online]. Available: https://doi.org/10.2514/1.40556.

[5] R. Palacios, "Nonlinear normal modes in an intrinsic theory of anisotropic beams", *Journal of Sound and Vibration*, vol. 330, no. 8, pp. 1772–1792, 2011, ISSN: 0022-460X. DOI: https://doi.org/10.1016/j.jsv.2010.10.023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0022460X10007078.

[6] X. Wang, W. Yang, and B. Sun, "Derivatives of kronecker products themselves based on kronecker product and matrix calculus", *Journal of Theoretical and Applied Information Technology*, vol. 48, pp. 378–382, Jan. 2013.

[7] M. Gosses, W. Nowak, and T. Wöhling, "Explicit treatment for dirichlet, neumann and cauchy boundary conditions in pod-based reduction of groundwater models", *Advances in Water Resources*, vol. 115, pp. 160–171, 2018, ISSN: 0309-1708. DOI: https://doi.org/10.1016/j.advwatres.2018.03.011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0309170817307467.

[8] M. Bakhtiari, A. A. Lakis, and Y. Kerboua, "Derivatives of fourth order kronecker power systems with applications in nonlinear elasticity", *Applied Mathematics and Computation*, vol. 362, p. 124 501, 2019, ISSN: 0096-3003. DOI: https://doi.org/10.1016/j.amc.2019.06.015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0096300319304801.

[9] A. del Carre, A. Muñoz-Simón, N. Goizueta, and R. Palacios, "Sharpy: A dynamic aeroelastic simulation toolbox for very flexible aircraft and wind turbines", *Journal of Open Source Software*, vol. 4, no. 44, p. 1885, 2019. DOI: 10.21105/joss.01885. [Online]. Available: https://doi.org/10.21105/joss.01885.

[10] B. Kramer and K. E. Willcox, "Nonlinear model order reduction via lifting transformations and proper orthogonal decomposition", *AIAA Journal*, vol. 57, no. 6, pp. 2297–2307, 2019. DOI: 10.2514/1.J057791. eprint: https://doi.org/10.2514/1.J057791. [Online]. Available: https://doi.org/10.2514/1.J057791.

[11] V. Khoromskaia and B. Khoromskij, *Tensor-based techniques for fast discretization and solution of 3d elliptic equations with random coefficients*, Jul. 2020. DOI: 10.48550/arXiv.2007.06524.

[12] C. Riso and C. Cesnik, "Correlations between um/nast nonlinear aeroelastic simulations and the pre-pazy wing experiment", Dec. 2020. DOI: 10.2514/6.2021-1712.

[13] M. Artola, "Minimal nonlinear modal aeroelastic descriptions for highly flexible aircraft control", Ph.D. dissertation, Imperial College London, 2021.

[14] M. Artola, *Minimal nonlinear modal aeroelastic descriptions for highly flexible aircraft control*, Aug. 2021. DOI: https://doi.org/10.25560/93366. [Online]. Available: http://hdl.handle.net/10044/1/93366.

[15] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*, 2nd ed. Cambridge University Press, 2022.

[16] N. Goizueta Alfaro, *Parametric reduced-order aeroelastic modelling for analysis, dynamic system interpolation and control of flexible aircraft*, Apr. 2022. DOI: https://doi.org/10.25560/100137. [Online]. Available: http://hdl.handle.net/10044/1/100137.

[17] P. Conti, M. Guo, A. Manzoni, A. Frangi, S. L. Brunton, and J. N. Kutz, *Multi-fidelity reduced-order surrogate modeling*, 2023. arXiv: 2309.00325 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2309.00325.

[18] R. Palacios and C. E. S. Cesnik, *Dynamics of Flexible Aircraft: Coupled Flight Mechanics, Aeroelasticity, and Control*. Cambridge University Press, 2023, ISBN: 978-1-108-42060-0. [Online]. Available: https://www.cambridge.org/core/books/dynamics-of-flexible-aircraft/E4136BC1E42C9C3837AA1E5595D2D93F.

[19] C. Dong and M. M. Arief, *Morphing wing designs in commercial aviation*, 2025. arXiv: 2502.07182 [eess.SY]. [Online]. Available: https://arxiv.org/abs/2502.07182.